

LIST TIPE 2 (DOUBLE LINKED LIST)

```
type ElmList2 = <Prev: address, Info: InfoType, Next: address>
type address = pointer to ElmList2
type List2 = <First: address>
```

Procedure CreateList(output L:List2)

```
{I.S.: -; F.S.: L list kosong}
```

Kamus lokal

-

Algoritma

```
First(L) <- NIL
```

Function IsEmptyList(L:List2) --> boolean

```
{mengembalikan true bila list kosong}
```

Kamus lokal

-

Algoritma

```
-> First(L) = NIL
```

Procedure PrintList(input L:List2)

```
{I.S. L terdefinisi; F.S.: -}
```

```
{menampilkan semua elemen list L}
```

Kamus lokal

P: address

Algoritma

```
P <- First(L)
```

```
while (P ≠ NIL) do
```

```
    output (info(P))
```

```
    P <- next(P)
```

```
{P = NIL}
```

Function NbElm(L:List2) --> integer

```
{menghitung banyaknya elemen list L}
```

Kamus lokal

P: address

Len: integer

Algoritma

```
P <- First(L)
```

```
Len <- 0
```

```
while (P ≠ NIL) do
```

```
    Len = Len + 1
```

```
    P = next(P)
```

```
{P = NIL}
```

```
-> Len
```

Procedure InsertVFirst(input/output L: List2, input V: sinfotype)

```
{I.S. List L mungkin kosong}
```

```
{F.S. P dialokasi, Info(P)=V}
```

```
{Insert sebuah elemen beralamat P dengan Info(P)=V sebagai elemen pertama list linier L yg mungkin kosong}
```

Kamus lokal

P: address

Algoritma

```
Alokasi(P)
```

```
if (P ≠ NIL) then
```

```
    info(P) = V
```

```
    if (First(L) = NIL) then
```

```
        First(L) <- P
```

```
        prev(P) <- NIL
```

```
        next(P) <- NIL
```

```
    else
```

```
        next(P) <- First(L)
```

```
        prev(P) <- NIL
```

```
        prev(First(L)) <- P
```

```
        First(L) <- P
```

Procedure InsertVLast(input/output L: List2, input V: infotype)

```
{I.S. List L mungkin kosong } □ { F.S. P dialokasi, Info(P)=V }
```

```
{Insert sebuah elemen beralamat P dengan Info(P)=V sebagai elemen akhir list linier L yg mungkin kosong}
```

Kamus lokal

P: address

Last: address

Algoritma

```
Alokasi(P)
```

```
if (P ≠ NIL) then
```

```
    if (First(L) = NIL) then
```

```
        InsertVFirst(L,V)
```

```
    else
```

```
        info(P) <- V
```

```
        next(P) <- NIL
```

```
        Last <- First(L)
```

```
        while (next(Last) ≠ NIL) then
```

```
            Last = next(Last)
```

```
        {next(Last) = NIL}
```

```
        prev(P) <- Last
```

```
        next(Last) <- P
```

Procedure DeleteVFirst(input/output L:List2, output V:infotype)

```
{I.S. List L tidak kosong}
```

```
{F.S. Elemen pertama list L dihapus, dan didealokasi. Hasil penghapusan disimpan nilainya dalam V. List mungkin menjadi kosong. Jika tidak kosong, elemen pertama list yang baru adalah elemen sesudah elemen pertama yang lama.}
```

Kamus lokal

P: address

Algoritma

```
P <- First(L)
```

```
if (P ≠ NIL) then
```

```
    V <- info(P)
```

```
    First(L) <- next(P)
```

```
    if (next(P) ≠ NIL) then
```

```
        prev(First(L)) <- NIL
```

```
    Dealokasi(P)
```

```

Procedure DeleteVLast(input/output L:List2, output V:infotype)
{I.S. List L tidak kosong}
{F.S. Elemen terakhir list L dihapus, dan didealokasi. Hasil penghapusan
disimpan nilainya dalam V. List mungkin menjadi kosong. Jika tidak
kosong, elemen terakhir list yang baru adalah elemen sebelum elemen
terakhir yang lama.}
Kamus lokal
  Last: address
Algoritma
  Last <- First(L)
  if (Last ≠ NIL) then
    while (next(Last) ≠ NIL) do
      Last <- next(Last)
    {next(Last) = NIL}
    V <- info(Last)
    if (prev(Last) = NIL) then
      Last <- NIL
    else
      next(prev(Last)) <- NIL
    Dealokasi(Last)

Procedure DeleteX(input/output L: List2, input X: infotype)
{I.S. List L tidak kosong}
{F.S. Elemen setelah X dihapus, dan didealokasi. List mungkin menjadi
kosong.}
Kamus lokal
  AX: address
  V: infotype
Algoritma
  if (First(L) ≠ NIL) then
    AX <- First(L)
    while (info(AX) ≠ X AND AX ≠ NIL) then
      AX <- next(AX)
    {info(AX) = X OR AX = NIL}
    if (info(AX) = X) then
      if (prev(AX) = NIL) then
        DeleteVFirst(L,V)
      else
        if (next(AX) = NIL) then
          DeleteVLast(L,V)
        else
          V <- info(AX)
          next(prev(AX)) <- next(AX)
          prev(next(AX)) <- prev(AX)
        Dealokasi(AX)

Procedure SearchX(input L:List2, input X:infotype, output A:address )
{I.S. L, X terdefinisi}
{F.S. A berisi alamat elemen yang nilainya X}
{Mencari apakah ada elemen list dengan info(P)= X. Jika ada, mengisi A
dengan address elemen tersebut. Jika tidak ada, A=Nil}
Kamus lokal
  P: Address
Algoritma
  A <- NIL
  P <- First(L)
  while (P ≠ NIL AND A = NIL) do
    if (info(P) = X) then
      A <- P
    P <- next(P)
  {P = NIL OR A ≠ NIL}

```

```

Procedure UpdateX(input/output L:List2, input X:infotype, input Y:infotype)
{I.S. L, X, Y terdefinisi}
{F.S. L tetap, atau elemen bernilai X berubah menjadi Y.
(Mengganti elemen bernilai X menjadi Y)}
Kamus lokal
  P: Address
Algoritma
  P <- First(L)
  while (P ≠ NIL) do
    if (info(P) = X) then
      Info(P) <- Y
    P <- next(P)
  {P = NIL}

procedure Invers(input/output L:List2)
{I.S. L terdefinisi, F.S. semua elemen L terbalik urutannya dari posisi
awal}
{membalik urutan posisi elemen list L}
Kamus lokal
  P: Address
  Lb: list2
Algoritma
  CreateList(Lb)
  Alokasi(P)
  P <- First(L)
  while (P ≠ NIL) do
    InsertVFirst(Lb, info(P))
    P <- next(P)
  {P = NIL}
  L <- Lb

Procedure InsertVAfterX(input/output L:List2, input X:infotype, input V:infotype)
{I.S. List L mungkin kosong}
{F.S. P dialokasi, Info(P)=V}
{Insert sebuah elemen beralamat P dengan Info(P)=V sebagai elemen dengan
posisi setelah elemen bernilai X}
Kamus lokal
  P: address
  AX: address
Algoritma
  if (First(L) ≠ NIL) then
    AX <- First(L)
    while (info(AX) ≠ NIL AND next(AX) ≠ NIL) then
      AX <- next(AX)
    {info(AX) = NIL AND next(AX) = NIL}
    if (info(AX) = X) then
      Alokasi(P)
      if (P ≠ NIL) then
        info(P) <- V
        next(P) <- next(AX)
        prev(P) <- AX
      if (next(AX) ≠ NIL) then
        prev(next(AX)) <- P
      next(AX) <- P

```

Procedure InsertVBeforeX(input/output L: List2, input X: infotype, input V: infotype)
 {I.S. List L mungkin kosong}
 {F.S. P dialokasi, Info(P)=V}
 {Insert sebuah elemen beralamat P dengan Info(P)=V sebagai elemen dengan posisi sebelum elemen bernilai X}

Kamus lokal

P: address
 AX: address

Algoritma

```

if (First(L) ≠ NIL) then
  AX ← First(L)
  while (info(AX) ≠ X AND next(AX) ≠ NIL) do
    AX ← next(AX)
  {info(AX) = X OR next(AX) = NIL}
  if (info(AX) = X) then
    Alokasi(P)
    if (P ≠ NIL) then
      info(P) ← V
      next(P) ← AX
      prev(P) ← prev(AX)
    if (prev(AX) ≠ NIL) then
      next(prev(AX)) ← P
    else
      First(L) ← P
  prev(AX) ← P

```

Procedure DeleteVAfterX(input/output L:List2, input X:infotype, output V:infotype)

{I.S. List L tidak kosong}
 {F.S. Elemen setelah X dihapus, dan didealokasi. Hasil penghapusan disimpan nilainya dalam V. □ List mungkin menjadi kosong.}

Kamus lokal

P: address
 AX: address

Algoritma

```

if (NbElm(L) ≥ 2) then
  AX ← First(L)
  while (info(AX) ≠ X AND next(AX) ≠ NIL) then
    AX ← next(AX)
  {info(AX) = X OR next(AX) = NIL}
  if (info(AX) = X) then
    if (next(AX) ≠ NIL) then
      P ← next(AX)
      V ← info(P)
      next(AX) ← next(P)
      next(P) ← AX
    Dealokasi(P)

```

Procedure DeleteVBeforeX(input/output L:List2, input X:infotype, output V:infotype)

{I.S. List L tidak kosong}
 {F.S. Elemen sebelum X dihapus, dan didealokasi. Hasil penghapusan disimpan nilainya dalam V. List mungkin menjadi kosong.}

Kamus lokal

P: address
 AX: address

Algoritma

```

if (NbElm(L) ≥ 2) then
  AX ← First(L)
  while (info(AX) ≠ X AND next(AX) ≠ NIL) do
    AX ← next(AX)
  {info(AX) = X OR next(AX) = NIL}
  if (info(AX) = X) then
    if (prev(AX) ≠ NIL) then
      P ← prev(AX)
      V ← info(P)
      if (prev(P) ≠ NIL) then
        next(prev(P)) ← AX
      prev(AX) ← prev(P)
    else
      if (P ≠ NIL) then
        prev(AX) ← NIL
      First(L) ← AX
    Dealokasi(P)

```

Function CountX(L:List2) -> integer

{mengembalikan banyaknya kemunculan X dalam list L}

Kamus lokal

P: Address
 Count: integer

Algoritma

```

P ← First(L)
Count ← 0
while (P ≠ NIL) do
  if (info(P) = X) then
    Count ← Count + 1
  P ← next(P)
{P = NIL}
-> Count

```

function Modus(L:List2) -> character

{mengembalikan huruf yang paling banyak muncul dalam list L}

Kamus lokal

P: Address
 count, max: integer
 modus: character

Algoritma

```

P ← First(L)
max ← 0
count ← 0
while (P ≠ NIL) do
  count ← CountX(L, info(P))
  if (count > max) then
    max ← count
    modus ← info(P)
  P ← next(P)
-> modus

```

```
Procedure SearchAllX(input L:List2, input X:infotype)
{I.S. L, X terdefinisi}
{F.S. -}
{Proses: menampilkan posisi-posisi kemunculan elemen X dalam list L}
Kamus lokal
  P: Address
  Count: integer
Algoritma
  Count <- 0
  P <- First(L)
  while (P ≠ NIL) do
    if (info(P) = X) then
      output (count)
      Count <- Count + 1
      P <- next(P)
  {P = NIL}
```